

Primary Use Cases:

- Create Account
- Login
- Logout
- Complete Daily Check-In
- View Pond
- Send Reminder/Ribbit
- Acknowledge Reminder/Ribbit
- Answer Daily Discussion Prompt
- Request Smart Connection Assistant Suggestion

Use case: CreateAccount

ID: UC-UA-01

Actors: PondMate/User

Preconditions:

- User has opened PondLite.
- User does not already have an account, or wants to create a new one.

Flow of events (Main Success Scenario):

1. User selects Create Account.
2. System displays the account creation form.
3. User enters required account information.
4. User submits the form.
5. System validates the submitted information.
6. System encrypts the user's password.
7. System creates and saves the new account.
8. System confirms account creation.
9. System redirects user to login, or automatically logs the user in.

Alternate flows:

A1 — Invalid or missing information

- **A1.1** At Step 5, system detects missing or invalid information.
- **A1.2** System displays an error message.
- **A1.3** User corrects the information and resubmits.
- **A1.4** Use case resumes at Step 5.

A2 — Account already exists

- **A2.1** At Step 5, system detects that the email/username is already associated with an account.

- **A2.2** System displays an account already exists message.
- **A2.3** User may choose to log in instead.

Postconditions:

- New user account is created and saved.
- User can now log in to PondLite.

Use case: Login

ID: UC-UA-02

Actors: PondMate/User

Preconditions:

- User has an existing account.
- User is not currently logged in.

Flow of events (Main Success Scenario):

1. User opens PondLite or selects Login.
2. System displays the login form.
3. User enters username/email and password.
4. User submits the login form.
5. System validates the submitted credentials.
6. System generates an authentication token/session.
7. System stores the active token/session.
8. System redirects user to the Pond/Home page.

Alternate flows:

A1 — Invalid credentials

- **A1.1** At Step 5, system detects that the username/email or password is incorrect.
- **A1.2** System displays an invalid login message.
- **A1.3** User may re-enter credentials and submit again.
- **A1.4** Use case resumes at Step 5.

A2 — Account does not exist

- **A2.1** At Step 5, system finds no account matching the submitted username/email.
- **A2.2** System displays an account not found message.
- **A2.3** User may choose to create an account.

Postconditions:

- User is logged in.
- User has an active token/session.

- User can access protected PondLite features.

Use case: Logout

ID: UC-UA-03

Actors: PondMate/User

Preconditions:

- User has an existing account.
- User is currently logged in.
- User has an active token/session.

Flow of events (Main Success Scenario):

1. User selects Logout.
2. System receives the logout request.
3. System invalidates or deletes the active token/session.
4. System clears the token/session from the client side.
5. System redirects user to the Login page.
6. System displays confirmation that the user has logged out.

Alternate flows:

A1 — Token/session already expired

- **A1.1** At Step 3, system detects that the token/session is already expired.
- **A1.2** System clears the token/session from the client side.
- **A1.3** System redirects user to the Login page.

Postconditions:

- User is logged out.
- User no longer has an active token/session.
- Protected PondLite features require login again.

Use case: CompleteDailyCheckIn

ID: UC-PL-01

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has not completed a check-in for the current day (or is allowed to edit it).

Flow of events (Main Success Scenario):

1. User enters the Pond page.

2. System checks if a daily check-in has been completed.
3. System prompts the user to complete their daily check-in.
4. System displays the emotion wheel.
5. User selects up to three emotions (primary required).
6. User optionally enters a message for their PondMate.
7. User submits the check-in.
8. System saves the check-in (emotions + optional message).
9. System updates the user's frog state based on the selected emotions.
10. System updates the Pond view.
11. System makes the check-in message available to the PondMate.

Alternate flows:

A1 — User skips optional message

- **A1.1** At Step 6, user chooses not to enter a message.
- **A1.2** System proceeds with emotion-only check-in.
- **A1.3** Use case continues at Step 7.

A2 — User already checked in

- **A2.1** At Step 2, system detects an existing check-in for the day.
- **A2.2** System either:
 - Prevents a second check-in and displays confirmation of completion, or
 - Allows the user to edit the existing check-in.
- **A2.3** If editing is allowed, flow resumes at Step 4.

Postconditions:

- User's daily emotional state is recorded.
- Frog state reflects the user's emotions.
- Optional message is available to the PondMate.
- System marks the user as "checked in" for the day.

Use case: ViewPond

ID: UC-PL-02

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has access to a Pond/relationship space.
- User has completed today's check-in, or the system allows limited pond access before check-in.

Flow of events (Main Success Scenario):

1. User selects or enters the Pond page.
2. System verifies that the user has a valid session.
3. System retrieves the user's Pond data.
4. System retrieves both PondMates' frog states.
5. System retrieves today's check-in status for both PondMates.
6. System retrieves any active reminders/ribbits, discussion prompts, or unread messages.
7. System displays the Pond screen.
8. System displays each frog's current emotional state and activity status.
9. System displays available Pond actions, such as check-in, reminders/ribbits, discussion, and AI suggestions.

Alternate flows:

A1 — User has not completed daily check-in

- **A1.1** At Step 5, system detects that the user has not checked in today.
- **A1.2** System prompts the user to complete the daily check-in.
- **A1.3** User completes the check-in.
- **A1.4** Use case resumes at Step 6.

A2 — No PondMate connected

- **A2.1** At Step 3, system detects that the user does not have a connected PondMate.
- **A2.2** System displays a message explaining that no PondMate is connected.
- **A2.3** System may provide an option to invite or connect with a PondMate.

Postconditions:

- User sees the current Pond state.
- User can view frog status, emotional reflection, and available connection actions.

Use case: SendReminder/Ribbit

ID: UC-PL-03

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has a connected PondMate.
- User is viewing the Pond or Reminder/Ribbit feature.

Flow of events (Main Success Scenario):

1. User selects Send Reminder/Ribbit.
2. System displays the reminder/ribbit creation form.
3. User chooses a reminder/ribbit type.

4. User enters a short message or selects a preset message.
5. User optionally selects a delivery time.
6. User submits the reminder/ribbit.
7. System validates the reminder/ribbit content.
8. System saves the reminder/ribbit.
9. System sends or schedules the reminder/ribbit for the PondMate.
10. System confirms that the reminder/ribbit was sent or scheduled.

Alternate flows:

A1 — Missing or invalid content

- **A1.1** At Step 7, system detects missing or invalid content.
- **A1.2** System displays an error message.
- **A1.3** User corrects the reminder/ribbit.
- **A1.4** Use case resumes at Step 6.

A2 — User cancels

- **A2.1** At any point before Step 6, user cancels the reminder/ribbit.
- **A2.2** System discards unsaved reminder/ribbit content.
- **A2.3** System returns user to the previous screen.

Postconditions:

- Reminder/ribbit is saved.
- PondMate can receive, view, or acknowledge the reminder/ribbit.

Use case: AcknowledgeReminder/Ribbit

ID: UC-PL-04

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has received a reminder/ribbit from their PondMate.
- Reminder/ribbit is active and not yet acknowledged.

Flow of events (Main Success Scenario):

1. User views the Pond or notifications.
2. System displays any active or unread reminders/ribbits.
3. User selects a reminder/ribbit.
4. System displays the reminder/ribbit details.
5. User selects Acknowledge.
6. System records the acknowledgment.

7. System updates the reminder/ribbit status to acknowledged.
8. System optionally notifies the sender that the reminder/ribbit was acknowledged.
9. System updates the Pond view to reflect the interaction.

Alternate flows:

A1 — Reminder already acknowledged

- **A1.1** At Step 2, system detects the reminder/ribbit has already been acknowledged.
- **A1.2** System displays the acknowledged status.
- **A1.3** No further action is required.

A2 — User dismisses without acknowledging

- **A2.1** At Step 5, user exits or dismisses the reminder/ribbit without acknowledging.
- **A2.2** System keeps the reminder/ribbit active/unacknowledged.
- **A2.3** Reminder remains available for later interaction.

Postconditions:

- Reminder/ribbit is marked as acknowledged (if completed).
- Sender can see that the reminder/ribbit was acknowledged.
- System updates relationship interaction state accordingly.

Use case: AnswerDailyDiscussionPrompt

ID: UC-PL-05

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has a connected PondMate.
- A daily discussion prompt is available.

Flow of events (Main Success Scenario):

1. User navigates to the Pond or Discussion section.
2. System displays the current daily discussion prompt.
3. User selects the prompt.
4. System displays an input area for the response.
5. User enters a response.
6. User submits the response.
7. System validates and saves the response.
8. System makes the response visible to the PondMate.
9. System updates the discussion status (e.g., “1/2 responses completed” or “Complete”).

Alternate flows:

A1 — User skips the prompt

- **A1.1** At Step 3, user chooses not to respond.
- **A1.2** System allows the user to return later.
- **A1.3** Prompt remains active for the day.

A2 — User edits response

- **A2.1** After Step 7, user selects to edit their response.
- **A2.2** System allows modification of the existing response.
- **A2.3** System saves the updated response.

A3 — Both users have responded

- **A3.1** At Step 9, system detects both PondMates have submitted responses.
- **A3.2** System marks the discussion as complete.
- **A3.3** System may highlight or surface the shared responses for both users to view.

Postconditions:

- User's response to the daily discussion prompt is saved.
- PondMate can view the response.
- System tracks discussion completion status for both users.

Use case: RequestSmartConnectionAssistantSuggestion

ID: UC-PL-06

Actors: PondMate/User

Preconditions:

- User is logged in.
- User has a connected PondMate.
- Relevant context exists (e.g., at least one recent check-in, reminder, or discussion activity).

Flow of events (Main Success Scenario):

1. User selects the Smart Connection Assistant feature.
2. System gathers relevant context:
 - User's current mood
 - PondMate's current mood (if available)
 - Recent check-in status
 - Active/unanswered reminders or ribbits
 - Recent discussion activity

3. System sends context to the AI service.
4. AI generates suggestions:
 - One care action
 - One message suggestion
 - One optional discussion prompt
5. System receives the AI response.
6. System displays the suggestions to the user.
7. User reviews the suggestions.
8. User may:
 - Accept and use a suggestion, or
 - Ignore the suggestions, or
 - Request regeneration
9. System records the interaction (optional, for future context improvement).

Alternate flows:

A1 — Insufficient context

- **A1.1** At Step 2, system detects limited or missing context.
- **A1.2** System proceeds with minimal available data.
- **A1.3** AI generates more general suggestions.
- **A1.4** Use case continues at Step 4.

A2 — AI request fails

- **A2.1** At Step 3, AI service is unavailable or request fails.
- **A2.2** System displays an error message.
- **A2.3** System allows the user to retry.

A3 — User regenerates suggestions

- **A3.1** At Step 8, user selects regenerate.
- **A3.2** System resends context to the AI service.
- **A3.3** AI returns new suggestions.
- **A3.4** System displays updated suggestions.

Postconditions:

- User receives optional AI-generated suggestions for connection.
- No actions are forced; suggestions remain user-controlled.
- System may store interaction data to improve future suggestions.

